

C++プログラミング (Part 2) *

担当: 平田耕一, 山本邦雄

E-mail: hirata@ai.kyutech.ac.jp, kunio@ai.kyutech.ac.jp

TA: 石丸嘉人, 磯部裕太

1 Apr. 2026

機材: 各自が所有する PC 上に構築した Linux 環境¹

注意: この実験・演習は第一部 Part 1 と第二部 Part 2 をそれぞれ 3 回, 合計 6 回で行う。関連科目の復習ほか事前学習を十分行ったうえでのぞむこと。

Part 2 では, グループ単位で演習を行いレポートを作成提出する。

1 大富豪/大貧民をプレイするソフトウェアの開発演習

ババ抜きで利用したトランプカードを表す型 Card とカードの集合の実装である CardSet を用いて, トランプ・ゲームの「大富豪 (大貧民)」をプレイする環境と思考ルーチンをそなえるプレーヤを作成し, オブジェクト指向言語によるモデル化にもとづくプログラミングとチームでのソフトウェア開発を実践する。

実現するシステム (体系) のよっている規則を完全に把握することが, コンピュータによるシステム構築に不可欠である。まず, 実現するゲームのルールを確認しよう。

2 大富豪/大貧民のルール

大富豪 (大貧民) は, ラミーや UNO と同様に, 場札の上に手札 (の組) を捨てていき, 手札をなくしたプレイヤーがあがりとなるゲームである。52 枚のカードとジョーカー 1 枚を使用する。

まず親 (ディーラー) は, カードをよくシャフルし, 大富豪から時計回りに一枚ずつ各プレイヤーに配る。すべてのカードを配り終えたら, ゲームを開始する。ターンは大貧民から, 反時計回りにまわっていく。ターンが回ってきたプレイヤーは, 手札から同じランク (数) のカードを一枚以上, ジョーカーを含め四枚まで組として場に出せる。このカードの組をリードという。ただし, 場にすでにリードがある場合は, 同じ枚数のより強いランクの札の組でなければ出すことはできない。リードがペア, 3 カード,

* 本資料は, 乃万 司, 下薮 真一, 嶋田 和孝, 中村 貞吾 によって作成されました。

¹ 自身の Windows PC に Linux 環境が入っていない学生は, ISC オンラインガイドの「仮想化アプリケーションの利用方法」(<https://onlineguide.isc.kyutech.ac.jp/guide2020/index.php/home/2020-02-04-07-29-54>) を見て環境を構築して下さい。

あるいは4カードならば、次のプレイヤは同じ枚数でより強いカードの組を出さねばならない。出せない、または、出したくない場合は、場に札を出さずにパスすることができる。場にカードを出すか、パスすればそのプレイヤのターンは終了である。ターンは、反時計回りにまわっていく。

カードのランクはジョーカーが最も強く、続いて $2 > A > K > Q > J > 10 > \dots > 3$ の順となる。ただし2枚以上の組にジョーカーを使った場合、ジョーカーは他のカードと同じランクであるとみなされる。全員がパスをつづけ、リードを出したプレイヤにターンが回ってきた場合、リードは「流れた」として場札はなくなったものとみなす。そして最後にカードを出した、つまりリードを出したプレイヤがリーダーとなる。リーダーは、リードとなる札（の組）を自由に選んで出すことができる。

最初に手札をなくしたプレイヤが大富豪、つまり1位となる。残るプレイヤも勝ち抜けるまでゲームをつづけ、全員の順位を決める。次のゲームは順位が低い方から反時計回りにならび、大貧民（最下位）が親となる。

今回は、これらを基本ルールとして、ゲームは基本ルールのみで行うものとする。大富豪のルールには多くのバリエーションやローカルルールがみられるが、たとえば次にあげるような点は実際にプレイされているルールと異なることになるので注意しよう。

- (1) ゲーム参加者数は8名以下とする。
- (2) 最後に出す札を制限しない。最後の札としてジョーカーや2を出してあがってもよい。
- (3) シークエンス（連続する複数のランクのカードからなる組もの、ポーカーでいうストレート）は、組としてみとめない。
- (4) 革命（4枚組、あるいは3の4枚組が出されると、それ以降カードのランクの強さが逆転し、強さが $3 > 4 > \dots$ の順となる）はない。
- (5) 大富豪の任意のカードと大貧民の強いカードの交換（搾取）、および大富豪が最初にあがれなかった場合に大貧民となるペナルティ（都落ち）は、採用しない。
- (6) 場札がなくなった（流れた等）状態でリードを取ったプレイヤはどんな札でも出せる状況にあるが、そこでもしパスした場合は、その次のプレイヤは（場が空なので）どんな札でも出すことができるものとする。場が空の状態にもかかわらず、各プレイヤがパスを続けてゲームが進行しなくなった場合は、一定回数経過後ゲームを終了させ、プレイ中であったプレイヤに対しては席の逆順に順位を割り当てる。

3 プログラミングのためのモデル：プレイヤの箱

プログラミングを行う場合、特にオブジェクト指向言語では、現実のものにあわせて、あるいは似せてデータ型（クラス）を作るのが最も効率的である。今回、大富豪ゲームを複数プレイヤが行う状況をプログラムにするにあたっては、次のような考え方で作ることにする。

1. ディーラーはゲームに参加せず、プレイヤたちとは独立した中立な立場でゲームをルールどおりに進行させる役割になう。
2. プレイヤは、ターンの回ってくる参加者。あがった後は、プレイヤではなくなる。

実際のカードゲームにおけるディーラーは、大貧民もしくはほかの誰かが行うが、このモデルではゲーム管理を行うクラスであり、実際にプレイはしない。すなわち、札を配るだけでなく、ターンなどルー

ルに従った行動をプレイヤに対してうながす、またプレイヤが出した札がリードとして通るかどうかなを確認し、また他のプレイヤに確認させる、などコミュニケーションを司る。ゲームに参加するプレイヤとその順位の管理も行う。つまり、ディーラーは万能で完全に公平なゲームの管理人である。

ゲームの進行は、カードのやりとり全てをディーラーに任せ、プレイヤは、「カードの出し入れができる引き出しがついた箱（に入っている人）」である。その箱の外には3つのスイッチがついており、ディーラーがそれぞれを押すと、箱の中で「ゲーム開始」、「あなたのターン」、「新しいリード」のランプが点灯する。

カードの配布は、ディーラーが引き出しにカードを入れ、「ゲーム開始」ランプを点灯させて終わる。ゲーム開始後、ディーラーはターンがきたプレイヤの引き出しにリードとゲーム状況を入れ、「あなたのターン」ランプを点灯させ、札を出すかパスするかを促す。引き出しにより強い組が入っていたら、新たなリードとして受け入れ、そうでなければ、カードはそのままにして引き出しを押し返す。そして他のプレイヤそれぞれに対して、「新しいリード」ランプを点灯させ、新たなリードを引き出しに入れて（あるいは空のまま）見せる。リードを出したプレイヤの手札がなくなった場合、ディーラーは勝ち抜けを記録する。また、勝ち抜けてプレー中でなくなったプレイヤには次回からターンをまわさない。

人間同士でゲームをする場合、これらの「手順」にかわるものは、言葉や視覚によって暗黙のうちに完成しているが、機械的なゲームの進行を実現するには、これらを手続きとして明確なものにしなければならない。たとえばネットワーク対戦を行うことを考えると、プレイヤが人であっても、機械であっても、このような手続きが必要であることが理解できるだろう。

4 基本となるソースコードと拡張の方法

この演習では、クラス継承のしくみを使い、基本となるプレイヤのクラスを継承の機能を用いて拡張することでオリジナルのプレイヤクラスを作る。まず、moodle で提供しているサンプルプログラム、ライブラリ、および、コンパイルを補助するファイルをダウンロードして準備する。

```
card.h, cardset.h, dealer.{h,cpp}, gamestatus.h,  
player.{h,cpp}, simpleplayer.{h,cpp}, defaultplayer.{h,cpp},  
daifugou.cpp, thinkTA1.h, libplayingcards.a, Makefile
```

コマンド `make` で各ファイルのコンパイルとリンクが行われ、実行ファイル `daifugou` ができるはずである。この実行ファイルと `.cpp` ファイルをコンパイルしてできたオブジェクトファイルは、`make clean` で削除できる。なお、`thinkTA1.h` は、過去の TA が作成したプレイヤクラス `ThinkTA1` を使用するためのヘッダファイルで、プログラムのソースコードは非公開だが、コンパイル済みオブジェクトは `Card`, `CardSet` クラスのオブジェクトとともにライブラリファイル `libplayingcards.a` として提供して、それをリンクして使用するようになっている。

クラス継承を使ったプレイヤクラスの拡張

具体的には、グループで作成する **拡張クラス (派生クラス)** で、`Player` クラスのメンバ関数のうち `ready()`, `follow()`, `approve()` をオーバーライド (上書き) する。たとえば、グループのプレイヤクラス名を `MyGroupPlayer` とするとしよう。ファイル `mygroupplayer.h` で

```
#include "player.h"

class MyGroupPlayer : public Player {
    ...
};
```

と宣言することで Player クラスを基底とするプレイヤクラスが作成できる。ゲーム進行に必要な Player クラスのメンバ変数やメンバ関数は、継承によってそのまま MyGroupPlayer クラスで利用できる。さらに、思考ルーチンに必要なデータ構造の定義や変数の宣言、関数の追加ができる。それらを用いて、ready(), follow(), approve() 関数（のうち必要なもの）を再定義して、オリジナルのプレイヤクラスを作成する。

注意： ダウンロードしたソースコードのうち、card.h, cardset.h, dealer.h, dealer.cpp, gamestatus.h, player.h, player.cpp, thinkTA1.h は変更してはならない。自分用のプレイヤは、Player クラスを変更するのではなく、それを継承した **派生クラス**（たとえば myGroupPlayer クラスなど）を新たに作成すること。（simpleplayer.{h,cpp} を参考にせよ）

Player には手札からランダムに一枚カードを出す follow 関数が実装されており、とりあえずゲームを進行させることが可能である。新たなプレイヤクラスでは、必要なデータ構造や手続きを作成し、follow() 関数、および必要に応じて他の 2 つの関数を書き換え、グループ独自のより強いプレイヤを作るのが目的となる。

作成したプレイヤクラスは、daifugou.cpp を

```
...
#include "player.h"
#include "simpleplayer.h"
#include "mygroupplayer.h"    <---- 追加
...
void registerPlayers(Dealer &d) {
    d.regist(new ThinkTA1("TA One"));
    d.regist(new Player("Pablo"));
    d.regist(new MyGroupPlayer("MyGroup"));  <---- プレイヤ入れ替え
    d.regist(new SimplePlayer("Simplet"));
}
...
```

のように変更し、ゲームに参加させる。なお、継承による拡張の例として SimplePlayer クラスを用意しているので、継承クラスの作り方の参考にしてほしい。

なお、新しく作ったプレイヤクラスを含めた形でコンパイルするには、Makefile を次のように修正するとよい。

```
HEADERS = card.h cardset.h dealer.h gamestatus.h player.h \
           simpleplayer.h defaultplayer.h thinkTA1.h mygroupplayer.h
OBJS = dealer.o player.o simpleplayer.o defaultplayer.o mygroupplayer.o
```

5 データ構造および関数

Player クラスでは、`ready()`、`follow()`、`approve()` の3つの関数を定義しているが、この関数自体は変更せずに、必ず派生クラスを作成して、同名の関数をオーバーライドして作成すること。

Player::ready() 関数

はじめてのゲームで、あるいは1ゲームが終了し新しいゲームが始まる際、カード配布後に一度だけ呼び出される。たとえば場に出た強いカードを記憶するなど、ゲームごとの初期化が必要な場合、この関数の中で行う。

形式	<code>virtual void Player::ready(void)</code>
引数	なし
返値	なし
呼出	カードの配布後、ゲームを開始する直前に <code>Dealer::newGame()</code> によって呼び出される。
作用	作成した内容による。

Player::follow() 関数

プレイヤーがターン中に行うアクションを定義する。主に、この `Player::follow()` 関数を派生クラスでオーバーライド（定義の上書き）することにより、各グループの思考ルーチンを実現する。引数は、ゲーム状況をあらわす構造体 (`GameStatus` 型) と、プレイヤーが出す札を入れるカードの集合である。返値は `true` であるが、現在つかわれていない。

形式	<code>virtual bool Player::follow(const GameStatus &gstat, CardSet &cards)</code>
引数	ゲームの状況 <code>gstat</code> とカード集合 <code>CardSet &cards</code> (実引数は空集合)
返値	論理値 <code>bool</code> 、ただし現在は使用されていない（予約済み値）。計算結果である「自分が出すカード」は <code>cards</code> に入れて返す。
呼出	<code>main</code> 関数のゲーム進行の中で、ターンがくるたび呼び出される。
作用	関数でセットされた引数 <code>cards</code> の内容は、場に出す札としてディーラーがチェックする。 <code>cards</code> に入れたカードは、自分の手札 (<code>hand</code>) から削除しておかなくてはならない。 <code>cards</code> が空である場合は、パスと判断される。空でなく、かつ新たなリードとして認められない場合、 <code>main</code> 関数のゲーム進行の中で、 <code>hand</code> に戻され、パスしたと判断される。

`cards` に出したカードは手札から削除しなければ、そのプレイヤーは手札が減らず、あがれない。また、手札にないカードを出したり、出したカード以外のカードを手札から削除したり、勝手に強いカードを手札に加えたりするのはイカサマとなる。

たとえば基底クラスの Player クラスでは、

```
bool Player::follow(const GameStatus &gstat, CardSet &cards) {
    CardSet pile(gstat.pile);    // 作業用に場札のカードセットをコピー
    Card tmp;
    hand.pickup(tmp, -1);        // 手札からランダムに一枚選ぶ
    cards.insert(tmp);           // それを出してみるだけ
    return true;                 // 常に true を返す
}
```

となっている。リードを一切考慮せず、自分の手持ちのカードセットからランダムに1枚出し、ディーラーの様子をうかがう、という動作である。

Player::approve() 関数

ターンが回ってきたプレイヤー以外の各プレイヤーが、場に出たカードセットを確認する機会をもうけるため、Player::approve() 関数が呼び出される。Player クラスでは何も行っていない。引数は、GameStatus 型データである。

形式	virtual bool approve(const GameStatus &gstat)
引数	ゲームの状況 gstat
返値	論理値 bool, ただし現在は使用していない。
呼出	ターンのプレイヤーが札を出した後、ターンのプレイヤー以外のプレイヤーに対して Dealer::showDiscardedToPlayers() の中で呼び出される。
作用	作成した内容による。

GameStatus 構造体

場札のほか、プレイヤーの並び、各プレイヤーの手札の枚数など、プレイ中に得ることができる情報をまとめた構造体である。プレイヤーは文字列型 std::string で表された名前では区別される。各フィールドの名称と意味は、以下のようになっている。

```
struct GameStatus {
    public:
        CardSet pile;           // 場札 (リード)
        int numPlayers;         // まだプレー中のプレイヤー数
        int numParticipants;    // (プレー中でないプレイヤーも含む) 全参加者数
        int turnIndex;          // ターンのまわってきたプレイヤーの添え字
        int leaderIndex;        // リーダーとなっているプレイヤーの添え字
        int numCards[MAXIMUM_NUM_OF_PLAYERS];
                                // 各プレイヤーが持つカード枚数: size() の結果
        int playerID[MAXIMUM_NUM_OF_PLAYERS];
                                // 各プレイヤー固有の ID 番号: id() の結果
        std::string playerName[MAXIMUM_NUM_OF_PLAYERS];
                                // 各プレイヤーの名前: playerName() の結果
        ...
};
```

内容を `std::cout << gstat << std::endl;` のように標準出力ストリームへ出力し確認することができる。

思考ルーチンを作成するうえでまず必要となるのは、pile フィールドであろう。内容へのアクセスは、CardSet の関数を用いて行う。CardSet クラスは、CardSet 同士での削除や追加ができるよう、また要素に添字によるアクセス `cards.at(i)` または `cards[i]` ができるよう拡張してある。拡張の内容については `cardset.h` のコメントを参照のこと。

turnIndex が指す位置は、プレイヤーの配列 (Dealer クラスのメンバ players) の添字である。この配列の各プレイヤーの情報は、GameStatus の numCards, playerID, playerName の同じ添字位置に入れ

られる。プレイ中のプレイヤは、この配列の先頭から `numPlayers - 1` までに入っており、`numPlayers` から `numParticipants - 1` までには、上がったプレイヤが入る（上がった順に、後から逆方向に並ぶ）。

ゲームが進行したり、新しいゲームになるとプレイヤの順番が入れ替わることがあるため、同じプレイヤを追跡したい場合は、プレイヤ ID (またはプレイヤ名) を利用するのがよい。

6 演習課題

演習 1

派生クラスで `follow()` をオーバーライドせよ。また必要があれば `approve()`, `ready()` を改良もしくは実装し、より知的な戦略に基づくプレイヤを実現せよ。

ただし、`daifugou.cpp` と `Makefile` のプレイヤ追加に必要な変更をのぞき、提供されたファイルはそのまま使用すること。

実装を考えるのは、オリジナルのクラスで定義するコンストラクタと、オーバーライドしてよい `ready()`, `follow()`, `approve()` , またこれらで使う関数や変数である。

7 グループ演習を進める上での注意

グループ演習では、リーダーを中心にグループ内で役割分担を決め、プログラムを作成すること。時間に限りがあるため、計画的に進めること。たとえば、まず、何らかの戦略に基づくプレイヤを複数実装する。複数のカード (ペアや 3 カードなど) を出せるようにしたり、選んだカードが場に通るかどうかを判定して出す、他のプレイヤの持ち札枚数を考慮する、などが考えられる。そして、それぞれを対戦させたり、考察することで、より洗練されたプレイヤに改良するプロセスを繰り返していく。

また、作業を進める中でソートやサーチなど、ツールとして必要な関数が出てくるだろう。そういったものを手分けして作成するのもよいだろう。

最終的には、各班で `Group` 班番号という名前の新たなプレイヤサブクラス (例えば 1 班なら、`Group1`) を作成し、大会に参加させる。班内でさまざまなアイディアを試す場合にも、戦略ごとにプレイヤクラスを定義して対戦させればよい。

実験の最後の時間に各班で作成した思考ルーチンを対戦させる。具体的には、ランダムに割り振られた予選リーグを行い、各予選リーグで成績優秀な班で決勝リーグを構成する。この順位は最終的な成績に加味する。

なお、大会の方式は、以下を予定している。

- `Group1` ～ `Group9` までの 9 チームによる競技会。
- 「3 チーム + TA + `SimplePlayer`」による予戦で、1 位が決勝進出、3 位は敗者復活戦 1 に、2 位が敗者復活戦 2 に回る。(順位は TA と `SimplePlayer` を除いた順位)
- 敗者復活戦 1 は、「3 チーム + TA + `SimplePlayer`」で行ない、1 位が敗者復活戦 2 に進出。
- 敗者復活戦 2 は、「4 チーム + TA」で行ない、1 位が決勝進出。
- 決勝は、「4 チーム + TA」で行なう。

- 1 試合は n ゲーム ($n = 5000 \sim 10000$ 程度). m チーム参加の 1 ゲームの順位に応じて得点を与える (1 ゲームの得点は, 1 位 $m + 1$ 点, 2 位 $m - 1$ 点, 3 位 $m - 2$ 点, m 位 1 点とする). 全ゲーム終了後, 合計得点の多い方から順に 1 位, 2 位, ... とする.

8 レポートについて

レポートの作成も全員が分担し, 以下の書式・期日で各班ごとに 1 つ提出すること.

1. A4 縦, 横書きとする. その他, Part1 のレポートに準ずる.
2. 表紙には, 演習名, 全班員の学生番号と名前を記載.
3. 各班員の役割分担や貢献内容を具体的に書くこと. 最終的に採用されなかった戦略なども含めて書いてよい.
4. 実装した思考ルーチンの説明 (工夫した点および不完全な点を含む).
5. 全体の考察とまとめ.

レポートは, このテーマの実験最終授業日の 2 週間後の 17 時までに, moodle の「後半レポート提出箱」にグループの代表者が提出する. また, 班ごとのレポートと同じ締切までに, 個人評価票 (書式指定) を各自提出すること. 提出先は「後半レポート提出箱」へ.